Appendix A: bash scripts used for file generation

The following scripts were used to generate test files of the required sizes. They were run in my MacOS computer's Terminal, and are written in the bash script language which Terminal uses.

Test 1

```
mkdir testfolder_1024x1
for i in {1..1024}; do
 dd if=/dev/zero of=testfolder_1024x1/file_$i.bin bs=1k count=1
done

mkdir testfolder_512x2
for i in {1..512}; do
 dd if=/dev/zero of=testfolder_512x2/file_$i.bin bs=2k count=1
done

mkdir testfolder_256x4
for i in {1..256}; do
 dd if=/dev/zero of=testfolder_256x4/file_$i.bin bs=4k count=1
done

mkdir testfolder_128x8
for i in {1..128}; do
 dd if=/dev/zero of=testfolder_128x8/file_$i.bin bs=8k count=1
done

mkdir testfolder_64x16
for i in {1..64}; do
 dd if=/dev/zero of=testfolder_64x16/file_$i.bin bs=16k count=1
done

mkdir testfolder_32x32
for i in {1..32}; do
 dd if=/dev/zero of=testfolder_32x32/file_$i.bin bs=32k count=1
done

mkdir testfolder_16x64
for i in {1..16}; do
 dd if=/dev/zero of=testfolder_16x64/file_$i.bin bs=64k count=1
done
```

```
mkdir testfolder_8x128
for i in {1..8}; do
 dd if=/dev/zero of=testfolder_8x128/file_$i.bin bs=128k count=1
done

mkdir testfolder_4x256
for i in {1..4}; do
 dd if=/dev/zero of=testfolder_4x256/file_$i.bin bs=256k count=1
done

mkdir testfolder_2x512
for i in {1..2}; do
 dd if=/dev/zero of=testfolder_2x512/file_$i.bin bs=512k count=1
done

mkdir testfolder_1x1024
for i in {1..1}; do
 dd if=/dev/zero of=testfolder_1x1024/file_$i.bin bs=1024k count=1
done
```

Test 2

```
dd if=/dev/zero of=./file_1mb.bin bs=1M count=1
```

Test 3

```
dd if=/dev/zero of=./file_10mb.bin bs=1M count=10
```

Appendix B: Python script to add a torrent file to multiple Transmission clients

This Python script uses “requests”, a library that can send requests to servers, to communicate with my Transmission clients, which are hosted locally on my computer. It then implements the Transmission remote control API to communicate with all my clients simultaneously and add the same torrent file to all of them, so they can start downloading. This is done to remove delay from manually adding torrent downloads to each client and ensuring consistent timing.

```
#Script that adds .torrent files to every client.
import requests
import base64
clients = [
    "http://localhost:9092/transmission/rpc",
    "http://localhost:9093/transmission/rpc",
    #...full list of clients here
]
#Specify name of file here
torrent_file = "files and torrents/file_5mb.torrent"
with open(torrent_file, "rb") as f:
    torrent_data = base64.b64encode(f.read()).decode()
for url in clients:
    #get session id
    r = requests.post(url)
    session_id = r.headers.get("X-Transmission-Session-Id")
    payload = {
        "method": "torrent-add",
        "arguments": {
            "metainfo": torrent_data
        }
    }
    headers = {"X-Transmission-Session-Id": session_id}
    r = requests.post(url, json=payload, headers=headers)
    print(f"{url}: {r.json()}")
```

Appendix C: Python script to see how long downloading took on a given client (used in Tests 1 and 3)

This Python script uses “requests”, a library that can send requests to servers, to communicate with my Transmission clients, which are hosted locally on my computer. It then implements the Transmission remote control API to get timing data from a client. Clients record when a download starts, how much is downloaded, and when a download is completed, which allows this script to either display a percentage of the file downloaded or display how long it took to download if the download is complete.

```
#Script to find time taken for Test 1.
import requests
import datetime
clients = [
    #Seed
    "http://localhost:9092/transmission/rpc",
    #Peer
    "http://localhost:9093/transmission/rpc",
]
for url in clients:
    #Get session ID
    r = requests.post(url)
    session_id = r.headers.get("X-Transmission-Session-Id")
    headers = {"X-Transmission-Session-Id": session_id}
    #Specify what to request from API
    payload = {
        "method": "torrent-get",
        "arguments": {
            "fields": ["name", "startDate", "doneDate", "percentDone", "isFinished"]
        }
    }
    r = requests.post(url, json=payload, headers=headers)
    data = r.json()["arguments"]["torrents"]
    print(f"\n{url}")
    for t in data:
        if t["percentDone"] >= 1.0:
            start = datetime.datetime.fromtimestamp(t["startDate"])
            done = datetime.datetime.fromtimestamp(t["doneDate"])
            duration = done - start
            print(f"{t['name']}: finished, took {duration} from {start} to {done}")
        else:
            print(f"{t['name']}: {t['percentDone']*100:.1f}% done")
```

Appendix D: Python script to see how long it takes all clients to download the file (used in Test 2)

This Python script uses “requests”, a library that can send requests to servers, to communicate with my Transmission clients, which are hosted locally on my computer. It then implements the Transmission remote control API to get timing data from a client. Clients record when a download starts, how much is downloaded, and when a download is completed.
This script collects this data from all the clients involved in the experiment, and displays the total time it took for all clients to download a file. It does this by finding the difference between when the first download starts and when the last download ends.

```
#Script to find time taken for Tests 2 and 3.
import requests
import datetime
from collections import defaultdict
clients = [
    #Peers only, not including seed
    "http://localhost:9093/transmission/rpc",
    "http://localhost:9094/transmission/rpc",
    "http://localhost:9095/transmission/rpc",
    #full list of clients here
]
# Store start and done times
torrent_times = defaultdict(lambda: {"starts": [], "dones": []})
for url in clients:
    r = requests.post(url)
    session_id = r.headers.get("X-Transmission-Session-Id")
    headers = {"X-Transmission-Session-Id": session_id}
    payload = {
        "method": "torrent-get",
        "arguments": {
            "fields": ["name", "startDate", "doneDate", "percentDone", "isFinished"]
        }
    }
    r = requests.post(url, json=payload, headers=headers)
    torrents = r.json()["arguments"]["torrents"]
    for t in torrents:
        if t["percentDone"] >= 1.0:
            if t["startDate"] and t["startDate"] > 0:
                torrent_times[t["name"]]["starts"].append((t["startDate"], url))
            if t["doneDate"] and t["doneDate"] > 0:
                torrent_times[t["name"]]["dones"].append((t["doneDate"], url))
```

```
print("\nTest results:")
for name, times in torrent_times.items():
    first_start_time, first_client = min(times["starts"], key=lambda x: x[0])
    last_done_time, last_client = max(times["dones"], key=lambda x: x[0])
    first_start_dt = datetime.datetime.fromtimestamp(first_start_time)
    last_done_dt = datetime.datetime.fromtimestamp(last_done_time)
    swarm_duration = last_done_dt - first_start_dt
    print(f"{name}:")
    print(f"  First start: {first_start_dt} on {first_client}")
    print(f"  Last done : {last_done_dt} on {last_client}")
    print(f"  Swarm duration: {swarm_duration}")
```

Appendix E: Docker Compose file used to create the containers

This is the .yaml configuration file used to create new containers running the required clients in Docker Compose. I used this to create 1 container running a tracker, and 21 containers running Transmission clients (the full code is shortened here, only showing the configuration for 1 of the Transmission clients, but the rest are identical except for their port numbers and IDs). I used this configuration file to set up the bridge network between them, allowing them to communicate directly on my computer hardware.

```
version: "3.6"
networks:
 torrent-network-1:
   driver: bridge
   driver_opts:
     parent: eth0
   ipam:
     config:
       - subnet: 192.168.1.0/24
         gateway: 192.168.1.1
services:
 #Torrent tracker
 tracker:
   image: tnhnclskn/bittorrent-tracker
   container_name: tracker
   ports:
     - "6969:6969"
   networks:
     - torrent-network-1
   restart: unless-stopped
 #One example of a container. A total of 21 of these were made, numbered sequentially.
 tm-container-01:
   container_name: "tm-container-01"
   entrypoint:
     - "/init"
   environment:
     - "PATH=/lsiopy/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
     - "PS1=$$(whoami)@$$(hostname):$$(pwd)\\$$ "
     - "HOME=/root"
     - "TERM=xterm"
     - "S6_CMD_WAIT_FOR_SERVICES_MAXTIME=0"
     - "S6_VERBOSITY=1"
```

```
      - "S6_STAGE2_HOOK=/docker-mods"
      - "VIRTUAL_ENV=/lsiopy"
      - "LSIO_FIRST_PARTY=true"
    hostname: "a55685b89173-1"
    image: "linuxserver/transmission:latest"
    ipc: "private"
    labels:
      build_version: "Linuxserver.io version:- 4.0.6-r4-ls311 Build-date:-
2025-09-23T10:39:34+00:00"
      maintainer: "aptalca"
      org.opencontainers.image.authors: "linuxserver.io"
      org.opencontainers.image.created: "2025-09-23T10:39:34+00:00"
      org.opencontainers.image.description:
"[Transmission](https://www.transmissionbt.com/) is designed\
       \ for easy, powerful use. Transmission has the features you want from a
BitTorrent client: encryption,\
       \ a web interface, peer exchange, magnet links, DHT, µTP, UPnP and NAT-PMP port
forwarding, webseed\
       \ support, watch directories, tracker editing, global and per-torrent speed
limits, and more."
      org.opencontainers.image.documentation:
"https://docs.linuxserver.io/images/docker-transmission"
      org.opencontainers.image.licenses: "GPL-3.0-only"
      org.opencontainers.image.ref.name: "df00bc19950c10334e194b6eb8bf6b3514030a5e"
      org.opencontainers.image.revision: "df00bc19950c10334e194b6eb8bf6b3514030a5e"
      org.opencontainers.image.source:
"https://github.com/linuxserver/docker-transmission"
      org.opencontainers.image.title: "Transmission"
      org.opencontainers.image.url:
"https://github.com/linuxserver/docker-transmission/packages"
      org.opencontainers.image.vendor: "linuxserver.io"
      org.opencontainers.image.version: "4.0.6-r4-ls311"
    logging:
      driver: "json-file"
      options: {}
    networks:
      - torrent-network-1
    ports:
      - "0:51414/tcp"
      - "0:51414/udp"
      - "9092:9091/tcp"
    volumes:
```

```
      - "/[DIRECTORY]/config:/config"
      - "/[DIRECTORY]/t1/downloads:/downloads"
      - "/[DIRECTORY]/t1/watch:/watch"
    working_dir: "/"
```

Appendix F: Python script to fit function to test data

This Python script that I created for Test 2 uses scipy, a data analysis library, to fit a parameter in my function to the data points I obtained from my test. It adjusts the parameter until a value is found that matches the data points as closely as possible, outputting the optimal value of the parameter. In this case, the parameter was the interpolation factor b, which indicates how much the growth rate is dependent on a linear factor and how much it is dependent on a logarithmic factor. It also calculated the RMSE (root mean square error), which is a measure of the average deviation of my data points from the function. A small deviation means my hypothesis is likely correct.

```
import numpy as np
from scipy.optimize import curve_fit


def lin(x): return 21*x + 11
def log(x): return 21*np.log2(x+1) + 11
def composite(x, b): return b*lin(x) + (1-b)*log(x)
x_data = np.array([1,3,5,7,9,11,13,15,17,19])
y_data = np.array([32.0,64.7,86.0,101.7,119.7,134.3,158.3,179.0,199.0,215.3])
popt, pcov = curve_fit(composite, x_data, y_data, p0=[0.5])
b = popt[0]
print(f"Best-fit b: {b}")

y_predicted = composite(x_data, popt)
rmse = np.sqrt(np.mean((y_data - y_predicted)**2))
print(f"RMSE: {rmse}")
```